



Contagem Paralela de Objetos em Matriz Binária

Sistemas Operacionais

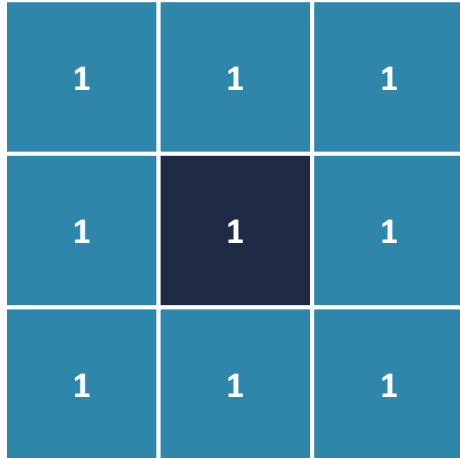
Nomes: Artur Schultz, Diego Schaeffer, Enzo Xavier e Rafael Menchik

Problema

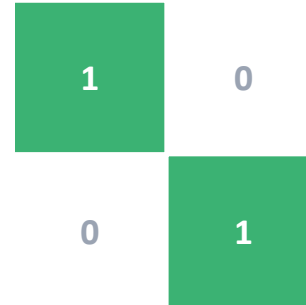
1	1	0	0	0
1	1	0	0	0
0	0	0	1	0
0	0	0	1	0
1	0	0	0	0

- 1 = pixel, 0 = fundo
- Objeto = grupo de 1s vizinhos
- Este exemplo: 3 objetos
- Duas versões (sequencial e paralela), mesmo resultado

Conectividade-8



Célula central: até 8 vizinhos



Diagonal: 1 objeto, não 2

(conectividade-4 daria 2)

Versão sequencial: flood fill com BFS

```
para cada célula (i, j):  
  se é 1 e não visitada:  
    contador++  
    BFS a partir de (i, j):  
      enfileira a semente  
      enquanto fila não vazia:  
        tira (r, c)  
        para cada vizinho-8 que é 1  
          e não visitado: marca e enfileira
```

- Fila iterativa no heap (tamanho $L \times C$), alocada uma vez
- BFS e não DFS: evita estouro de pilha (requisito 41)
- Cada célula processada uma vez: $O(L \times C)$ em tempo e espaço

Versão paralela: visão geral

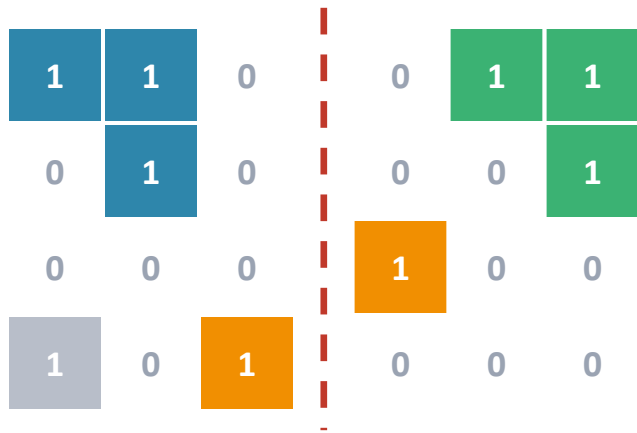
1 Divisão Main divide em $BR \times BC$ blocos e reserva labels

2 Rotulagem local N threads: BFS dentro de cada bloco (paralelo)

3 Consolidação Main une labels vizinhos na DSU (sequencial)

4 Contagem Conta raízes distintas = n° de objetos

Fronteiras e Union-Find



Esquerda: 3 componentes | Direita: 2

Soma ingênua = 5 → correto = 4

(3,2) e (2,3): mesmo objeto cortado

- Cada label começa como seu próprio grupo
- Vizinhos com labels diferentes: unir
- Nº de grupos no fim = nº de objetos
- find(x): representante · unir(a, b): junta grupos
- Path compression + union by rank: $\sim O(1)$

Sincronização

Ponto	Mecanismo
Pegar próximo bloco	1 mutex (só incrementa um índice)
Escrever labels	Nenhum: blocos são disjuntos
Alocar labels	Nenhum: range reservado antes das threads
DSU	Nenhum: roda em uma única thread

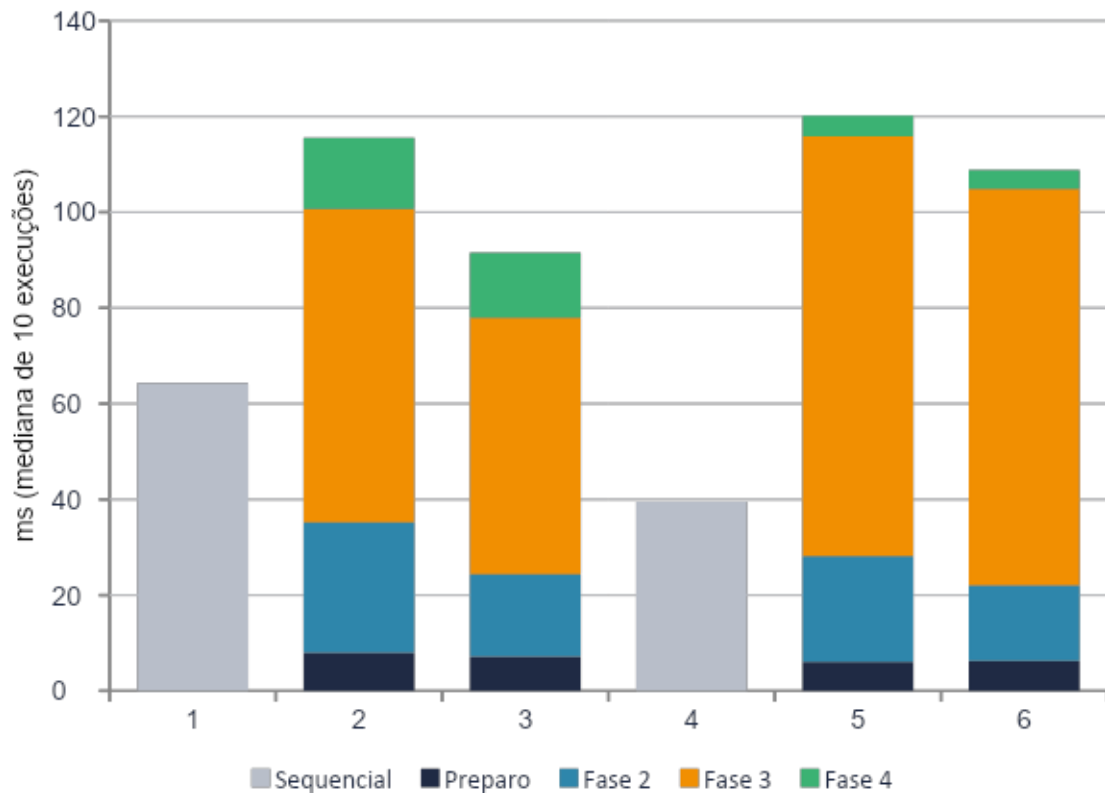
Sem race condition · sem deadlock (nenhum lock aninhado)

Testes

Arquivo	Dim.	Esperado	Seq.	Paralelo
m1_5x5	5x5	3	✓	✓
m2_6x8	6x8	4	✓	✓
m3_8x8	8x8	5	✓	✓
m4_9x12	9x12	6	✓	✓
m5_12x12	12x12	7	✓	✓
tabela1_10x10	10x10	5	✓	✓
tabela2_10x10	10x10	4	✓	✓
tabela3_10x10	10x10	6	✓	✓

- Várias combinações de threads e grades
- 1200×1200 aleatória: mesmo resultado nas duas versões (51.284 objetos)
- C89, -Wall -Wextra -pedantic: sem warnings

Desempenho



- Fase 2 escala com as threads
- Fase 3 (sequencial) domina: 58% a 75% do tempo
- 8 threads: +43% (fragmentada) e +179% (1 componente) vs. sequencial

Grade 16×16 (256 blocos) · i7-1255U (10 núcleos, 12 threads)

Conclusão e trabalho futuro

- Duas versões equivalentes e corretas
- Fase 2 paralela escala; consolidação sequencial limita o ganho (Amdahl)

Trabalho futuro

- Consolidar só as fronteiras entre blocos: $O(L \times C) \rightarrow O(\text{perímetro})$
- Eliminar a Fase 4: somar componentes locais e subtrair 1 a cada união efetiva
- Variar a grade $BR \times BC$ (2×2 , 4×4 , 8×8)